

CAHIER DES CHARGES FONCTIONNEL ET TECHNIQUE

PROJET TENYGUARD

Système d'Intelligence Artificielle pour la Détection Automatique de Gros Mots et Propos Vulgaires en Langue Malagasy dans les Vidéos

Intitulé du Projet :	TenyGuard — Détecteur de Vulgarités Vidéo (Malagasy)
Nature du Document :	Cahier des Charges Spécifique & Référentiel Technique
Technologies Clés :	Python 3.11 / FastAPI (POO) / PostgreSQL / FFmpeg / LLM
Architecture Logicielle :	Programmation Orientée Objet (POO) & Clean Architecture
Version du Document :	1.0 (Version Finale Validée)
Date de Rédaction :	Août 2026
Statut du Projet :	Initialisation Terminée — Prêt pour Développement

Note d'orientation stratégique : Ce document constitue le cahier des charges fonctionnel et technique officiel du projet **TenyGuard**. Il formule la problématique du traitement automatique de la langue malagasy, délimite le périmètre applicatif, détaille l'architecture POO en 4 niveaux de détection, et présente la matrice comparative des fonctionnalités MVP vs Version Finale.

TABLE DES MATIÈRES

1. Présentation & Problématique du Projet	Page 3
1.1 Contexte du numérique à Madagascar	Page 3
1.2 Problématique de la modération en langue Malagasy	Page 3
1.3 Vision stratégique et objectifs du système	Page 3
2. Périmètre du Projet & Délimitation Applicative	Page 4
2.1 Périmètre applicatif et médias pris en charge	Page 4
2.2 Inclusions (Fonctionnalités couvertes)	Page 4
2.3 Exclusions (Hors périmètre de la v1)	Page 4
3. Architecture & Stack Technique	Page 5
3.1 Choix de la stack technique (Python, FastAPI, POO, Postgres)	Page 5
3.2 Architecture logicielle en couches (Clean Architecture)	Page 5
3.3 Pipeline global de traitement vidéo-to-detection	Page 5
4. Moteur de Détection Hybride à 4 Niveaux	Page 6
4.1 Niveau 1 : Dictionnaire de termes vulgaires malagasy	Page 6
4.2 Niveau 2 : Moteur de normalisation textuelle	Page 6
4.3 Niveau 3 : Analyse contextuelle par LLM	Page 6
4.4 Niveau 4 : Score de confiance hybride et sévérité	Page 6
5. Matrice Comparative : Version MVP vs Version Finale (Full)	Page 7
5.1 Tableau récapitulatif détaillé des fonctionnalités	Page 7
6. Spécifications des API REST & Modèle Relationnel	Page 8
6.1 Définition des endpoints REST (FastAPI)	Page 8
6.2 Structure des réponses JSON (Timestamps & Confiance)	Page 8
6.3 Modèle relationnel PostgreSQL	Page 8
7. Exigences Non-Fonctionnelles	Page 9
7.1 Performance et temps de réponse	Page 9
7.2 Scalabilité et modularité de l'architecture POO	Page 9
7.3 Sécurité, confidentialité et conformité	Page 9
8. Roadmap, Planning & Modalités de Recette	Page 10
8.1 Planning prévisionnel d'exécution (12 semaines)	Page 10
8.2 Critères d'acceptation du MVP & Signatures	Page 10

RÉSUMÉ EXÉCUTIF

Le projet **TenyGuard** répond à un besoin critique d'assainissement et de modération automatique des contenus vidéos diffusés en langue malagasy. Combinant le traitement automatique de la parole (Speech-to-Text) et une approche hybride de modération de texte (Dictionnaire algorithmique + Analyse contextuelle par Intelligence Artificielle / LLM), TenyGuard permet d'identifier précisément les instants exacts (timestamps) où des propos injurieux ou vulgaires sont prononcés.

1. PRÉSENTATION & PROBLÉMATIQUE DU PROJET

1.1 Contexte Général du Numérique à Madagascar

L'explosion de la création de contenus multimédias (réseaux sociaux, plateformes d'hébergement vidéo, podcasts, médias en ligne) à Madagascar a entraîné une diffusion massive de vidéos en langue malagasy. Cependant, cette démocratisation s'accompagne d'une augmentation de contenus véhiculant des insultes, des vulgarités, du harcèlement verbal et des discours offensants.

1.2 Problématique Spécifique de la Langue Malagasy

La modération automatique des contenus malagasy se heurte à plusieurs défis majeurs inexistants dans les langues très dotées :

- **Rareté des outils NLP dédiés** : Absence de modèles commercialisés de détection de vulgarités pré-entraînés pour le malagasy.
- **Variabilité de la langue orale vs écrite** : Contractions fréquentes, argot urbain, expressions régionales et dialectales.
- **Camouflage orthographique intentionnel** : Altération volontaire de l'écriture des mots vulgaires (remplacement de lettres, répétitions de voyelles, voyelles muettes).
- **Ambiguïté contextuelle** : Mots ayant un sens littéral neutre mais devenant vulgaires ou insultants selon le contexte de la phrase.

1.3 Vision Stratégique & Objectifs

L'objectif de TenyGuard est de fournir une solution logicielle industrielle et souveraine, capable de transcrire et d'analyser automatiquement les vidéos en malagasy pour en extraire un rapport horodaté complet des infractions verbales.

Objectif 1 : Précision Temporelle	Conserver les timestamps exacts (début/fin) de chaque terme problématique.
Objectif 2 : Hybridation IA	Allier la rapidité d'un dictionnaire algorithmique et la finesse contextuelle d'un LLM.
Objectif 3 : Architecture POO	Concevoir en Python POO modulaire sans dépendances de conteneurs complexes.

2. PÉRIMÈTRE DU PROJET & DÉLIMITATION

2.1 Périmètre Applicatif et Médias Pris en Charge

TenyGuard cible le traitement de fichiers vidéos stockés ou téléversés via une API REST. Le système extrait la bande sonore audio sans altérer la vidéo originale et traite le flux sonore de façon optimale.

2.2 Inclusions (Périmètre Couvert par le Projet)

- ✓ **Réception et validation des vidéos** : Prise en charge des formats standards (MP4, AVI, MOV, MKV).
- ✓ **Extraction Audio automatique** : Pipeline FFmpeg optimisé pour isoler l'audio au format mono WAV (16kHz).
- ✓ **Speech-to-Text (STT) Malagasy** : Conversion de la voix en texte avec génération de timestamps par mot/phrased.
- ✓ **Normalisation Textuelle avancée** : Traitement des majuscules, répétitions de lettres, diacritiques et caractères spéciaux.
- ✓ **Détection Hybride à 4 niveaux** : Dictionnaire de termes vulgaires + Analyse contextuelle LLM pour les termes ambigus.
- ✓ **Restitution des résultats structurés** : Génération de réponses JSON avec niveau de sévérité, score de confiance et timestamps.
- ✓ **Stockage persistant PostgreSQL** : Conservation de l'historique des vidéos, transcriptions et résultats de détection.

2.3 Exclusions (Hors Périmètre de la Version Initiale)

- ✗ **Modération de flux vidéo Live en temps réel** : Le système v1 traite des fichiers enregistrés en différé.
- ✗ **Bip sonore / Masquage vidéo automatique** : Pas de ré-encodage vidéo avec altération de la piste audio dans la v1.
- ✗ **Reconnaissance visuelle d'images ou gestes obscènes** : L'analyse porte exclusivement sur la piste audio et la parole.
- ✗ **Déploiement Docker** : Conformément aux contraintes, l'exécution se fait directement sur l'environnement hôte Python.

3. ARCHITECTURE & STACK TECHNIQUE

3.1 Stack Technique Retenue

Composant	Technologie	Rôle & Justification
Langage	Python 3.11+	Langage principal, idéal pour le NLP et les API.
Framework API	FastAPI	Framework REST asynchrone ultra-rapide avec Swagger auto.
Traitement Audio	FFmpeg	Extraction audio haute performance depuis la vidéo.
Speech-to-Text	Whisper-Mg	Transcription de la parole en texte malagasy avec timestamps.
Analyse LLM	OpenAI / Local LLM	Levée d'ambiguïtés et analyse du contexte de la phrase.
Base de données	PostgreSQL	Stockage relationnel robuste des vidéos, transcriptions et résultats.
ORM	SQLAlchemy 2.0	Gestion POO des entités et requêtes base de données.
Validation	Pydantic v2	Validation stricte des schémas de données et configurations.
Testing	Pytest	Tests unitaires et d'intégration automatisés.

3.2 Architecture Logicielle Orientée Objet (POO)

Le projet suit les principes de conception POO (Clean Architecture & Separation of Concerns). Les contrôleurs d'API (`routes`) ne contiennent aucune logique métier et délèguent les traitements aux services (`services`), qui s'appuient eux-mêmes sur des moteurs de détection spécialisés (`detectors`) et la couche d'accès aux données (`repositories`).

4. MOTEUR DE DÉTECTION HYBRIDE (4 NIVEAUX)

Afin d'optimiser à la fois la latence, les coûts d'appel LLM et la précision globale, TenyGuard déploie un système de détection en 4 niveaux successifs :

NIVEAU 1 : Dictionnaire Vulgaire (`DictionaryDetector`)

Recherche directe de correspondances exactes parmi un lexique prédéfini de termes et insultes malagasy. Temps d'exécution quasi-instantané ($O(1)$ via Hash Set).

NIVEAU 2 : Moteur de Normalisation (`TextNormalizer`)

Transformation du texte avant analyse : mise en minuscules, suppression des diacritiques/accents, réduction des répétitions abusives de lettres (ex: 'boooza' -> 'boza'), nettoyage de la ponctuation.

NIVEAU 3 : Analyse Contextuelle LLM (`ContextDetector`)

Intervention d'un modèle de langage pour évaluer les phrases ambiguës ou le second degré. Détermine si une expression neutre devient insultante selon l'intention de l'interlocuteur.

NIVEAU 4 : Score de Confiance Hybride (`HybridDetector`)

Aggregation des résultats des niveaux 1 à 3 pour calculer un score de confiance global entre 0.0 et 1.0, attribuer une catégorie (insult, profanity, offensive) et déterminer le niveau de sévérité (low, medium, high).

5. MATRICE COMPARATIVE : MVP VS VERSION FINALE

Ce tableau détaille l'arbitrage fonctionnel entre la version **MVP (Minimum Viable Product)** et la version **Finale (Full Release)** de TenyGuard.

Domaine / Fonctionnalité	Version MVP (Initiale)	Version Finale (Full Release)
Traitement Vidéo & Audio	Extraction audio locale synchronisée avec FFmpeg (format WAV mono). Validation basique du format.	File d'attente asynchrone (Celery/Redis), découpage par segments courts (chunking) et gestion des très gros fichiers.
Speech-To-Text (STT)	Modèle STT Whisper-Mg standard pour l'obtention des textes et timestamps au niveau des segments.	Fine-tuning du modèle STT sur les accents régionaux malagasy et alignement au mot près (word-level timestamps).
Dictionnaire de Gros Mots	Lexique de base des injures et vulgarités malagasy courantes (termes statiques).	Base dynamique gérable via API/Admin, gestion des synonymes dialectaux et détection de leet speak.
Analyse Contextuelle LLM	Appels LLM (OpenAI API) sur les segments identifiés comme suspects ou ambigus.	Modèle LLM local fine-tuné spécifiquement pour le malagasy afin de supprimer la dépendance aux API externes.
Stockage PostgreSQL	Tables relationnelles fondamentales (Vidéos, Transcriptions, Détections).	Indexation avancée, stockage des métadonnées enrichies, historique d'audit et statistiques globales.
API & REST Endpoints	Endpoints `/health`, `/videos/upload`, `/detection/analyze` fonctionnels.	Authentification JWT/OAuth2, gestion des quotas, webhooks de notification de fin de traitement.

6. SPÉCIFICATIONS DES API & MODÈLE DE DONNÉES

6.1 Endpoints REST FastAPI

- **GET /health** : Vérification de l'état opérationnel du service et de la base PostgreSQL.
- **POST /api/v1/videos/upload** : Téléversement d'un fichier vidéo, enregistrement et lancement de l'extraction audio.
- **GET /api/v1/videos/{id}** : Consultation de l'état d'avancement du traitement d'une vidéo.
- **POST /api/v1/detection/analyze** : Déclenchement de l'analyse de détection sur la transcription horodatée.

6.2 Format JSON des Résultats de Détection

```
{ "video_id": "9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d", "total_detections": 1, "detections": [ { "text":  
"expression vulgaire détectée dans le contexte", "term": "terme_normalisé", "category": "insult",  
"severity": "high", "confidence": 0.94, "start_time": 92.4, "end_time": 93.1 } ] }
```

6.3 Modèle Relationnel PostgreSQL

Le schéma PostgreSQL s'articule autour des tables principales :

- **videos** : id (UUID), filename, filepath, duration, status, created_at.
- **transcriptions** : id, video_id (FK), full_text, segments (JSONB).
- **detections** : id, video_id (FK), term, category, severity, confidence, start_time, end_time.

7. EXIGENCES NON-FONCTIONNELLES

7.1 Performance & Temps de Réponse

- Le temps total de traitement d'une vidéo (extraction + STT + détection) ne doit pas excéder **2 fois la durée réelle de la vidéo**.
- L'endpoint d'analyse sur un texte déjà transcrit doit répondre en **moins de 500 ms** pour les correspondances dictionnaire.

7.2 Scalabilité & Modularité POO

- L'architecture orientée objet doit permettre le remplacement d'un modèle STT ou LLM par simple implémentation d'une classe dérivée.
- L'utilisation d'interfaces abstraites (`BaseProfanityDetector`) garantit l'extensibilité sans régression du code existant.

7.3 Sécurité & Confidentialité

- Les vidéos et fichiers audio téléversés doivent être conservés dans un répertoire sécurisé avec droits d'accès restreints.
- Nettoyage automatique ou archivage configurable des fichiers temporaires audio WAV après traitement.

7.4 Qualité du Code & Maintenabilité

- Couverture de tests unitaires et d'intégration avec Pytest d'au moins **80%**.
- Respect strict des normes PEP 8, du typage Python (Type Hints) et validation par Pydantic v2.

8. ROADMAP, PLANNING & MODALITÉS DE RECETTE

8.1 Planning Prévisionnel par Phases (12 Semaines)

Phase	Durée	Livrables Principaux
Phase 1 : Setup & Architecture	Semaines 1-2	Initialisation FastAPI, POO, PostgreSQL, CI/CD GitHub Actions.
Phase 2 : Extraction & STT	Semaines 3-5	Pipeline FFmpeg, intégration Speech-to-Text malagasy et timestamps.
Phase 3 : Moteur Hybride	Semaines 6-8	Dictionnaire malagasy, normalisation, intégration du LLM contextuel.
Phase 4 : Tests & Optimisation	Semaines 9-10	Tests d'intégration pytest, optimisation des temps de réponse.
Phase 5 : Recette & Livraison	Semaines 11-12	Recette fonctionnelle finale, documentation OpenAPI, déploiement.

8.2 Critères d'Acceptation du MVP

Le projet sera déclaré validé lorsque les critères suivants seront satisfaits :

- Succès de l'extraction audio et de la transcription sur un jeu d'essai de 10 vidéos de test.
- Détection exacte des termes vulgaires du dictionnaire avec tolérance de timestamp < 1 seconde.
- Validation de l'API REST via la documentation Swagger interactive et exécution conforme des CI/CD.

8.3 Signatures & Approbation

Pour le Maître d'Ouvrage (Client / Chef de Projet)	Pour l'Équipe de Développement
Date : ____ / ____ / 2026	Date : ____ / ____ / 2026
Signature :	Signature :